



SOFT2201/COMP9201 Assignment 2

Code Refactoring

1 Background

In this assignment, you are asked to complete three partially scaffolded refactorings of an existing question bank and quiz generation application. The purpose of this assignment is to help you gain experience in reading complex but realistic code and refactoring code with design patterns. You have three tasks:

1. First, we have to make the eligibility policies composable and renderable with the Composite pattern.
2. Second, we have to support the creation of a new importer based on JSON Lines (JSONL) with the Factory Method pattern.
3. Third, we have to support undo operations using the Command pattern.

More instructions about the assignment can be found on Ed: <https://edstem.org/au/courses/36363/lessons/117940/slides/810615>.

2 Assignment Description

Refer to the Python repository accompanying this PDF document. The assignment provides two versions of the codebase:

- `before_refactoring/` is a runnable version of the code *before* refactoring. It can be helpful for understanding the program's behaviour.
- `src/` is the partially completed version of the code *after* refactoring, with scaffolding for the above tasks.

To complete your tasks, please edit the code in `src/`, and NOT `before_refactoring/`. Fill in the missing code marked by the placeholders `??? TODO`. The placeholders do not necessarily occupy just one line. You may add as much code as needed to replace them. Some test cases are provided in `tests/`. You can add more tests if you think it'll help you in your tasks, but do not submit them.

3 Setup

After completing the relevant placeholders, you can run the application with the following commands:

```
export PYTHONPATH=src
python -m quiz_platform.question_bank.cli import \
    data/questions.json
python -m quiz_platform.question_bank.cli import \
    data/questions.csv
python -m quiz_platform.question_bank.cli import \
    data/questions.jsonl
python -m quiz_platform.question_bank.cli candidates \
    data/questions.json --request data/request.json \
    --reference-date 2026-09-25
python -m quiz_platform.question_bank.cli interactive \
    data/questions.json --request data/request.json \
    --reference-date 2026-09-25 --simple
```

The development dependencies are optional. However, if you wish to run the tests or use the shorter `quiz-bank` command, you have to install the additional dependencies. You can do so with the following commands:

```
python -m pip install -e ".[dev]"
pytest
```

4 Tasks

Complete the three partially scaffolded refactorings under `src/` by filling the ??? `TODO` placeholders with your answers.

1. Composite Eligibility Rules (40 points).

- (a) First, read `core.py` under `before_refactoring/src/` and understand the code. Next, compare both `core.py` and `rules.py` under `src/`. Pay attention to the classes extending `EligibilityRule` as well as the ??? `TODO` placeholders for Question 1.
- (b) Complete the Composite refactoring by implementing `AllOfRule.exclusion_reason` and `AllOfRule.render`. The Composite must stop at and return the first exclusion reason produced by its children in their declared order. It must recursively render its children as a parenthesised equation joined with `AND`; an empty Composite renders as `TRUE`.
- (c) Implement `ReuseCooldownRule.exclusion_reason`. A question with `last_used:null` is eligible. If `last_used` is not `None`, parse the value as a `YYYY-MM-DD` date, and compare it with the rule's reference date stored in `self._reference_date`. Exclude questions that were used less than the requested number of days ago. However, a question used exactly the requested number of days ago should not be excluded and is eligible. If the `last_used` date is invalid, raise a `ValueError`.
- (d) Fill the three placeholders in `build_candidate_pool`. This should construct the rule tree with the `rules_from_request` function, record its rendered expression in the report, and evaluate the questions using that rule tree.

Check and use the test cases in `tests/test_rules.py`.

2. Factory Method Importer Creation (30 points).

- (a) First, compare `importers.py` under `before_refactoring/src/` with the partially implemented `importers.py` under `src/`. The before client directly selects and constructs JSON or CSV importer Products. Complete the Factory Method refactoring by implementing the supplied ??? TODO placeholders. Check and use the test cases in `tests/test_importers.py`.
- (b) Complete the implementation of `create_importer` for the Concrete Creators such that each method returns the appropriate `QuestionImporter`. Implement `JsonLinesQuestionImporter.import_file` to pass the records to the `_complete_import` function. In a JSONL file, each non-empty line can be parsed as a JSON object. Non-object records should result in `ValueError`. Do not modify the existing behaviour for JSON/CSV files.

3. Command Moderation and Undo (30 points).

- (a) First, read `session.py` under `before_refactoring/src/`. Then compare it with `commands.py` and `session.py` under `src/`.
- (b) Complete the Command refactoring by filling in the ??? TODO placeholders in `commands.py` and `session.py`. Complete `_ChangeStatusCommand.execute` and `_ChangeStatusCommand.undo`. When undoing a command, the previous status should be restored, and the command should be marked as undone. Complete `QuestionBankSession.can_undo` such that it returns `True` if there is a command available to undo. The application's code uses it to display a `Nothing to undo.` message when the history is empty.
- (c) Implement `QuestionBankSession.execute` so it executes a command and adds it to the command history only after successful execution. Complete the two placeholders in `QuestionBankSession.undo_last`: remove and delegate to the latest command, and restore it to the history if undo raises an exception. For the expected command behaviour, check and use the test cases in `tests/test_commands.py`.

4. Optional Questions (0 points).

These questions are optional and are not worth marks. There is no word limit, but we appreciate concise answers. The teaching team will aggregate anonymised responses and discuss them in class.

Write your answers in `answers.md`.

- (a) Which design pattern(s) did you find difficult to distinguish from a simpler implementation? What made this distinction difficult for you?
- (b) Can you find code that may be a violation of the Don't Repeat Yourself (DRY) principle in the codebase? If so, where?

5 Submission

Submit the files listed below through Canvas: <https://canvas.sydney.edu.au/courses/73772/assignments/694868>.

Before submitting your work, you may want to search for `TODO` in the code and check that you have replaced every placeholder with your answer. However, please ensure you're submitting files from the `src` directly (NOT the `before_refactoring/` directory).

On Canvas, submit only these files:

1. `src/quiz_platform/question_bank/core.py`,
2. `src/quiz_platform/question_bank/rules.py`,
3. `src/quiz_platform/question_bank/importers.py`,
4. `src/quiz_platform/question_bank/commands.py`,
5. `src/quiz_platform/question_bank/session.py`, and
6. `answers.md` if you answered the optional questions. Otherwise, do NOT submit this file.

6 Grading Rubric

Question 1 (40 points)

- **12 points** for implementing `AllOfRule` such that it correctly evaluates and recursively renders children. It should return the first exclusion reason.
- **8 points** for correct reuse cooldown time duration evaluation, including `null` and invalid `last_used` validation.
- **10 points** for constructing the rule tree and storing its rendered expression through the two corresponding `build_candidate_pool` integration points.
- **10 points** for evaluating each question through the rule tree, and correctly updating the eligible questions and exclusion report.

Question 2 (30 points)

- **8 points** for Concrete Creators that return the appropriate Product.
- **7 points** for the JSONL Concrete Product. It should handle empty/blank lines and validate that each non-empty line contains a JSON object.
- **8 points** for connecting the Creator's shared operation to `create_importer` and preserving its import-and-validation workflow.
- **7 points** for preserving existing behaviour related to JSON/CSV imports.

Question 3 (30 points)

- **10 points** for completing command execution and undo.
- **10 points** for having the session execute commands, storing ONLY successful commands, and accurately reporting history availability through `can_undo`.
- **6 points** for undo that delegates to command objects without using an action/status conditional.
- **4 points** for ensuring that the command history remains accurate and consistent.